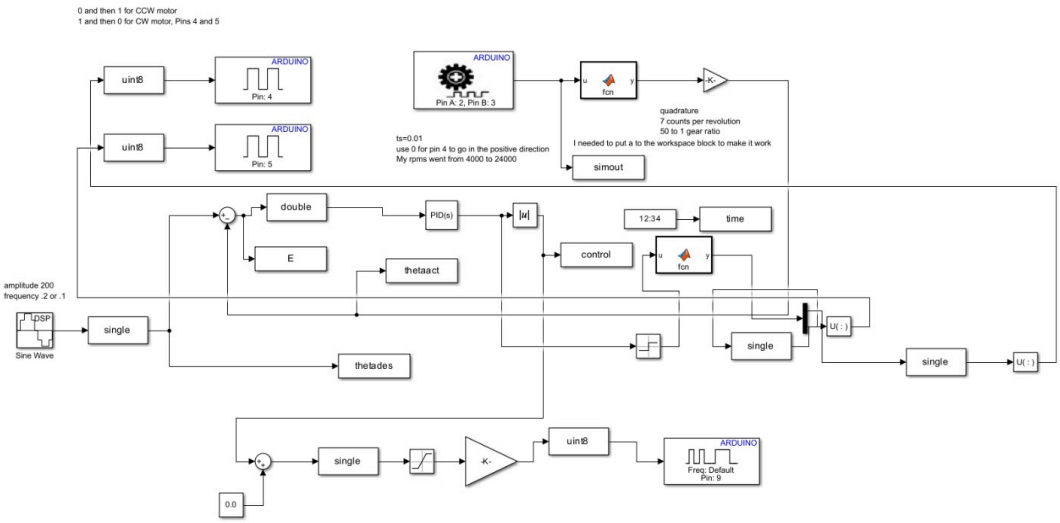
Analysis of the Simulink Motor Control Model

Abhijit Sinha , ASUID: 1234397959

Goal:The goal of this project is to design and implement a closed-loop DC motor position control system on a microprocessor platform using Simulink for model-based design and real-time code generation. The system will use a quadrature encoder for continuous position feedback and focus on systematically tuning P, PD, and PID controllers to achieve accurate trajectory tracking, reduced steady-state error, and improved stability and transient response.

Simulink Model:



Block Diagram Overview

The Simulink model is organized as a closed-loop position control system around a DC motor. At the top level, it can be split into the following main blocks:

1. **Command / Reference Generation**
2. **Measurement and Encoder Interface**
3. **Error Generation**
4. **Controller (P, PD, PID)**
5. **Motor Drive and Direction Logic**
6. **Arduino Target and Execution Settings**

Each block is made up of smaller sub-blocks, as described below.

1. Command / Reference Generation Block

This block defines what position the motor should try to follow.

- **Reference Generator (Sine Wave block)**
  - Produces a slowly varying sinusoidal position command.
  - The amplitude and frequency are chosen so that the motor can physically track the signal without saturating.
- **Scaling / Sign Blocks (Gain, Invert, etc.)**
  - Adjust the reference signal so that it is expressed in the same units and sign convention as the feedback (e.g., encoder counts or degrees).

Function: Supplies the desired trajectory θ_{ref} that will be compared against the measured position.

2. Measurement and Encoder Interface Block

This block converts raw encoder signals into a usable position measurement.

- **Quadrature Encoder Interface (Arduino Encoder block)**
  - Reads the A and B channels from the encoder connected to the Arduino pins.
  - Internally handles the counting and direction detection, providing a cumulative count value.
- **Conversion / Mapping (Function or Fcn block)**
  - Transforms encoder counts into a physical position, such as degrees or revolutions, using the counts-per-revolution and gear ratio.
- **Optional Gain / Offset**
  - Used to adjust the scale or flip the sign of the measured position so that it aligns with the control convention.

Function: Produces the actual motor position θ_{meas} that is fed back to the controller.

3. Error Generation Block

Here the difference between what we want and what we have is computed.

- **Subtractor Block**
  - Takes the reference position from Block 1 and the measured position from Block 2.
  - Computes the control error:
    $e(t)=\theta_{ref}(t)-\theta_{meas}(t)$

Function: Provides the error signal that drives the controller and reflects how well the motor is tracking the command.

4. Controller Block (P, PD, PID)

This block shapes the system response and is the core of the study.

- **PID Controller Block**
    - A single configurable block that can act as:
        - **P controller** (only Kp active),
        - **PD controller** (Kp and Kd active, Ki =0),
        - **Full PID controller** (Kp ,Ki , and Kd all active).
    - Gains are tuned experimentally or analytically to study effects on overshoot, settling time, and steady-state error.
- **Output Conditioning (Absolute Value / Saturation / Scaling)**
    - Ensures the control output is within allowable bounds for the PWM hardware (e.g., 0–255).
    - The absolute-value block is often used before PWM so the duty cycle is always non-negative.

Function: Generates a control effort based on the error that will be applied to the motor through the power electronics.

5. Motor Drive and Direction Logic Block

This part translates the controller output into electrical signals for the motor.

- **PWM Generation (Analog Output / PWM block)**
    - Converts the controller output magnitude into a duty cycle for a PWM pin on the Arduino.
    - The duty cycle determines how much average voltage is applied to the motor (hence its speed/torque).
- **Direction Logic (Digital Output blocks)**
    - Two digital pins determine the rotation direction (e.g., forward or reverse) through an H-bridge or motor driver.
    - Simple logic is used: if the original controller output is positive, set one pin high and the other low; if negative, reverse them.

Function: Implements both **how hard** the motor is driven (PWM duty cycle) and **which way** it spins (direction pins).

6. Arduino Target and Execution Block

This block is not always visible as a single icon, but it is critical for deployment.

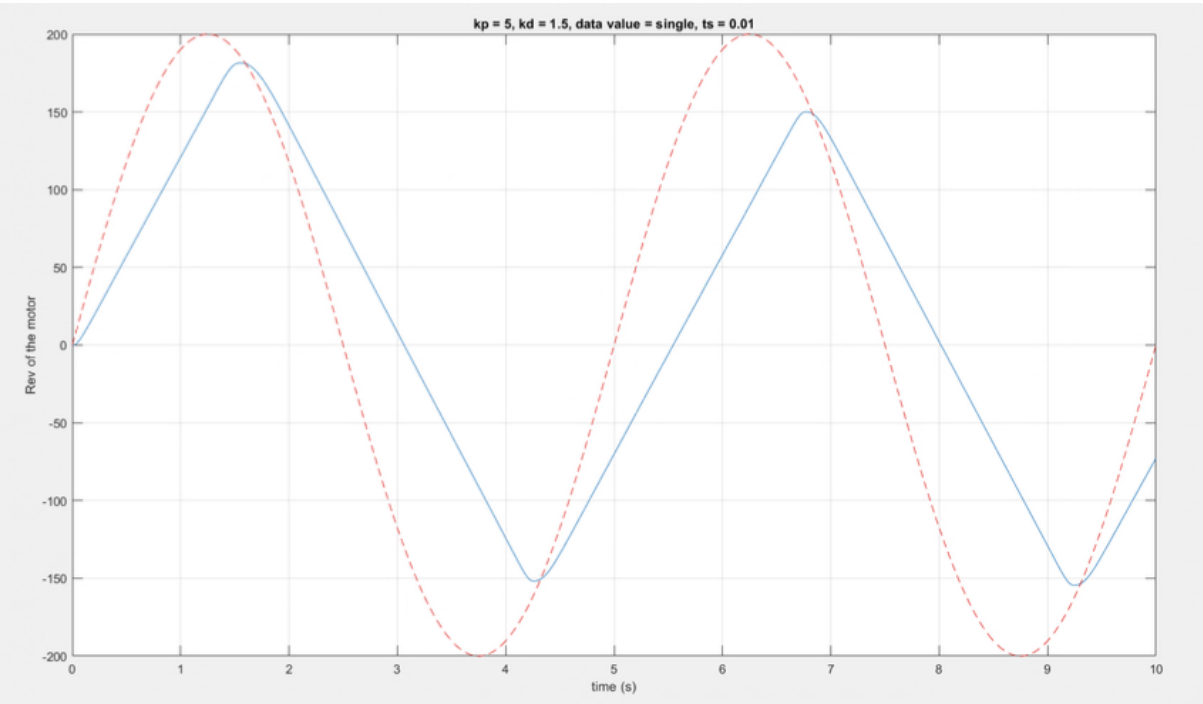
- **Hardware Configuration**
    - Specifies the Arduino board type, communication settings, and I/O pin assignments for encoder, PWM, and direction pins.

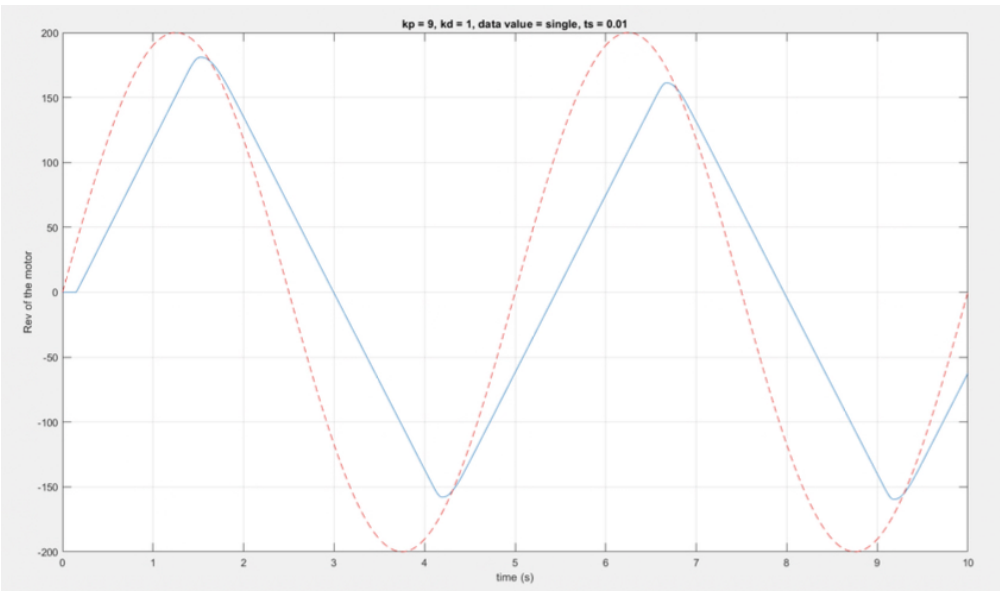
- **Solver and Step Size Settings**
    - Defines the sampling time for running the control loop in real time on the microcontroller.
- **Code Generation Settings**
    - Enables automatic conversion of the Simulink model into matlab code that is compiled and flashed onto the Arduino.

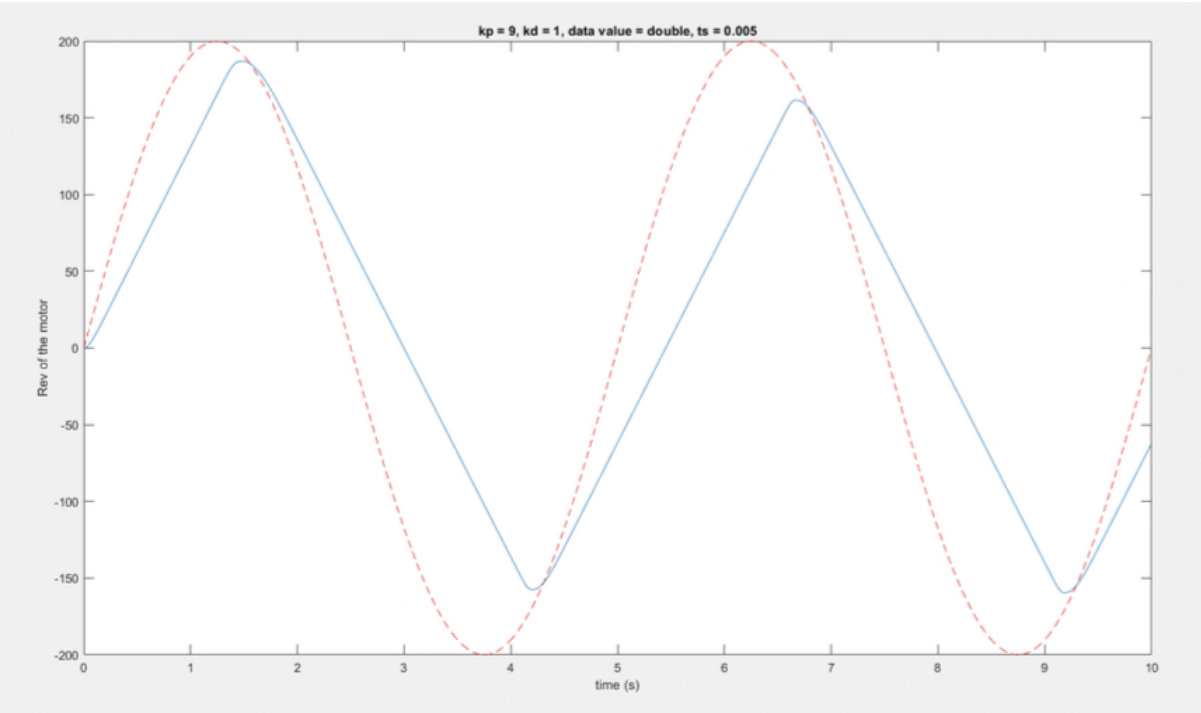
Function: Bridges the gap between the Simulink model and the physical hardware, allowing real-time closed-loop experiments.

Results:

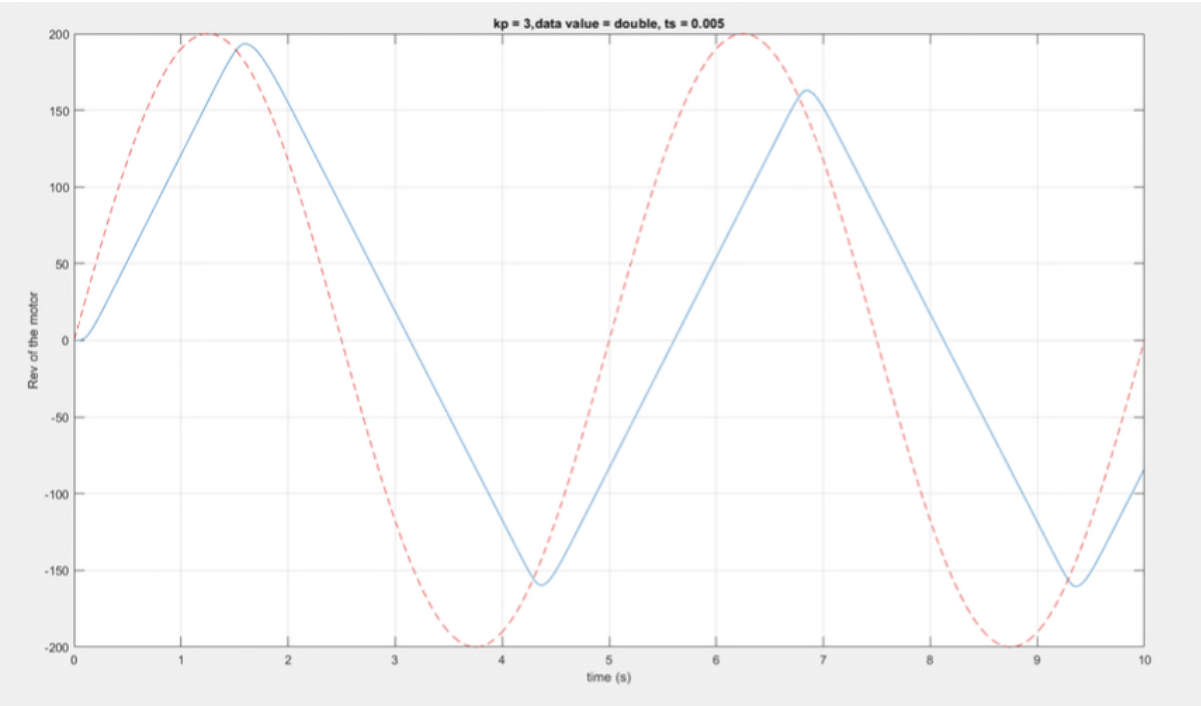
Diagrams with pd control:

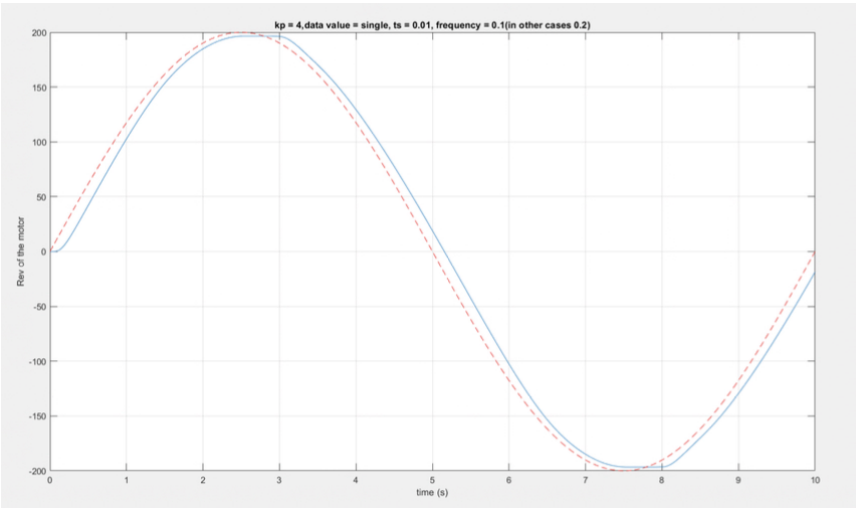






Diagrams with P control:





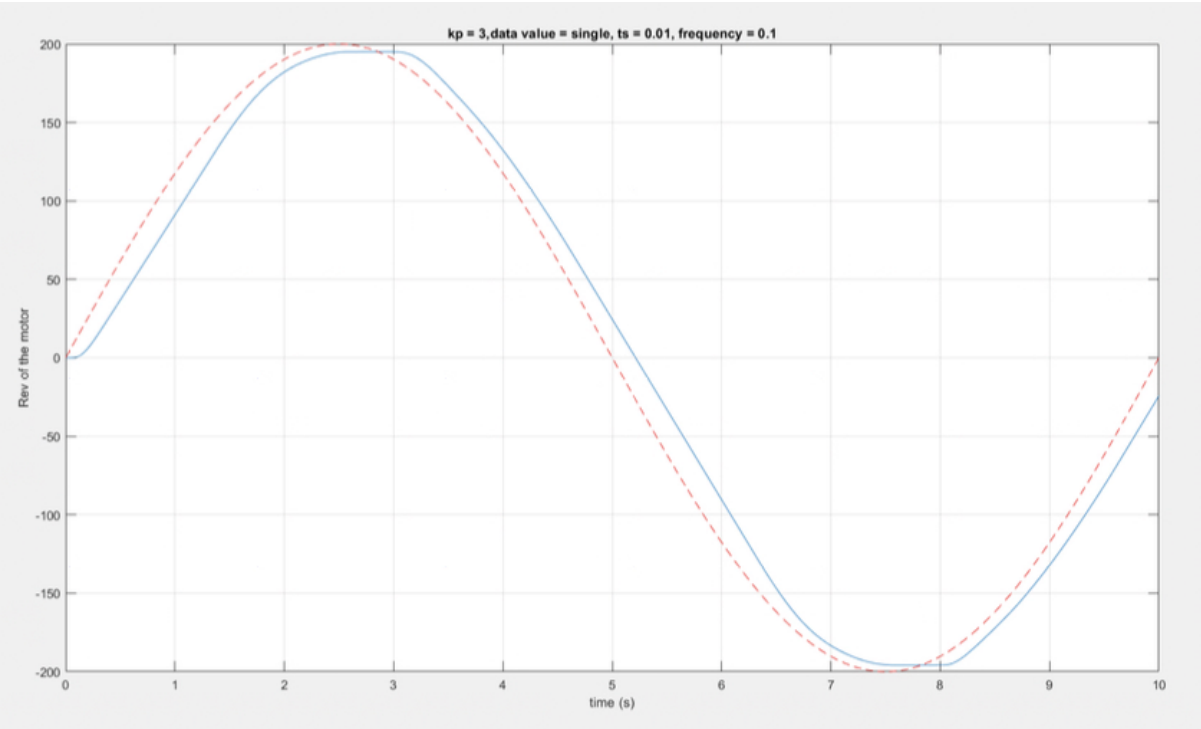
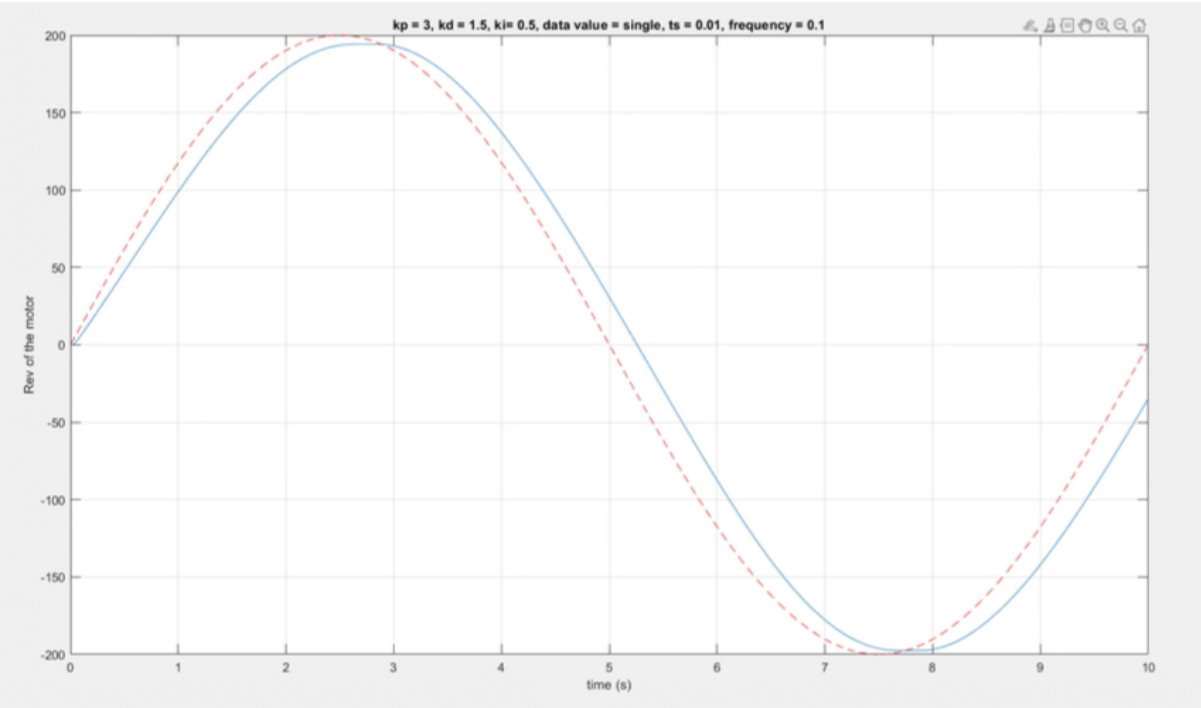
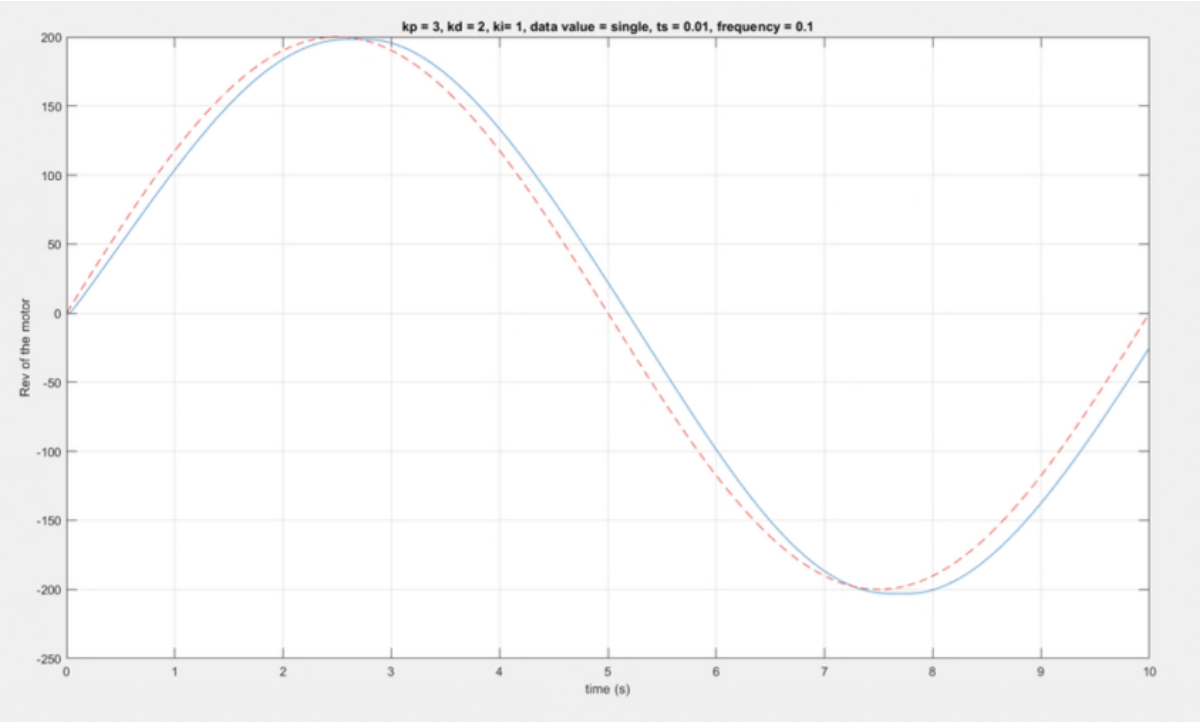
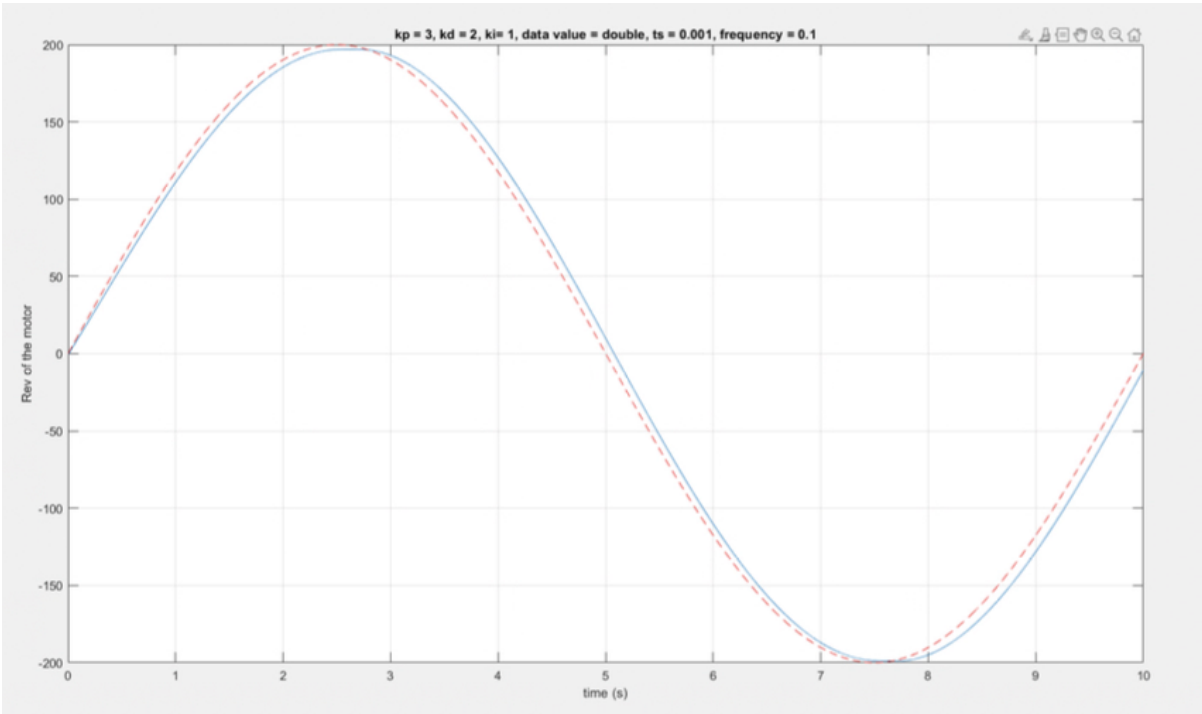


Diagram with PID control:





slight overshoot in negative region



Conclusion

In conclusion, this project will implement a complete closed-loop position control system for a DC motor using Simulink and an Arduino-based microcontroller, with real-time feedback from a quadrature encoder. By systematically tuning and comparing P, PD, and PID controllers on the same hardware platform, the work will quantify how derivative and integral actions influence tracking accuracy, steady-state error, and transient behavior. The expected outcome is a well-validated

experimental setup and a clear performance comparison between the three control strategies, providing both practical insight into PID tuning and a deeper understanding of microprocessor-based motion control.

Google Drive link for video

https://drive.google.com/file/d/1UWXcHq8Mjw7CbyEyw01xvqpLJDn5b8WS/view?usp=sharing